

장애·중복·권한 경계를 검증한 백엔드 시스템

주문 상태 수렴, 동시성 불변식, 권한 검색, LLM 비용 보호, CDC 계보, 추천 최신성을 코드와 반복 가능한 시나리오로 확인했습니다.

Spring Boot

Python API

Kafka

PostgreSQL

Redis/RabbitMQ

RAG/LLM

트랜잭션 · 권한 · 동시성

상태 수렴과 업무 불변식 보호

LLM · 데이터 · 개인화

정책 파이프라인과 데이터 품질 추적

01 StockRush

재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남

동일 이벤트 2회에도 outbox 1건, 늦은 결제 승인 무시, relay 5회 실패 뒤 FAILED

Spring Boot

Kafka

PostgreSQL

04 AI Gateway

앱마다 중복되는 LLM 정책과 provider 장애 처리

8단계 파이프라인과 5개 라우팅 모드, JSON·SSE·401·400 경로를 회귀 테스트로 고정

Java 21

Spring Boot WebFlux

Redis

03 Member Event Consistency

중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액

동시 8건→보상 1건, 60원 차감 2건→잔액 40, 용량 3→정확히 3건 발급

Spring Boot

PostgreSQL

Redis

05 CDC Data Platform

중복 CDC, 복구 이력 손실, 연결되지 않은 구간의 보장 과장

raw 7건 중 6건 적용·중복 1건 억제, 4개 source table과 mart lineage 추적

Spring Boot

PostgreSQL

Debezium

02 Enterprise Policy RAG

비인가 문서 노출과 근거 없는 답변 생성

문서 5개 중 허용 3개만 검색, 근거 0이면 answer=null-provider 호출 0회

Python API

React/Vite

PostgreSQL/pgvector

06 Fashion Personalization Platform

중복·실패 이벤트가 프로필과 추천 snapshot을 오염함

동일 key 1회 반영, 3회 실패 뒤 DLQ, 6개 상품 ranking과 snapshot 2개 이상·stale 0

Python 3.11+

optional FastAPI

pytest

01

StockRush

분산 커머스 / 주문 상태 수렴 / 이벤트 기반 백엔드

- 문제** 재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남
- 설계** Saga 상태 전이, 트랜잭션 Outbox, 먹등 소비, 종료 상태 SQL guard
- 결과** 동일 이벤트 2회에도 outbox 1건, 늦은 결제 승인 무시, relay 5회 실패 뒤 FAILED

04

AI Gateway

LLM Gateway / 라우팅 / 비용 보호 / 캐시 / 폴백 / 관측

- 문제** 앱마다 중복되는 LLM 정책과 provider 장애 처리
- 설계** API key-quota, 2단계 cache, 정책 router, 제한된 retry-fallback, trace
- 결과** 8단계 파이프라인과 5개 라우팅 모드, JSON-SSE-401-400 경로를 회귀 테스트로 고정

03

Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

- 문제** 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액
- 설계** DB 제약·조건부 갱신을 최종 guard로 두고 Redis-RabbitMQ를 경합별 비교
- 결과** 동시 8건→보상 1건, 60원 차감 2건→잔액 40, 용량 3→정확히 3건 발급

05

CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

- 문제** 중복 CDC, 복구 이력 손실, 연결되지 않은 구간의 보장 과장
- 설계** 독립 runtime-control plane-lakehouse 실험, event ID, ledger, replay 상태
- 결과** raw 7건 중 6건 적용·중복 1건 억제, 4개 source table과 mart lineage 추적

02

Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

- 문제** 비인가 문서 노출과 근거 없는 답변 생성
- 설계** 검색 전 권한 필터, citation 재검사, 근거 없음 refusal, 질의 로그
- 결과** 문서 5개 중 허용 3개만 검색, 근거 0이면 answer=null-provider 호출 0회

06

Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 batch / 이벤트 처리

- 문제** 중복·실패 이벤트가 프로필과 추천 snapshot을 오염함
- 설계** 먹등 event processor, retry-DLQ, 결정적 ranking, batch snapshot
- 결과** 동일 key 1회 반영, 3회 실패 뒤 DLQ, 6개 상품 ranking과 snapshot 2개 이상-stale 0

📁 상태·불변식 모델링

복잡한 동시 처리 문제를 상태 전이와 업무 불변식으로 분해하고 경계마다 보호 장치를 선택했습니다.

불변식 정의 → 보호 장치 선택 → 동시성 검증

적용 결과

주문 Saga와 outbox로 서비스 간 상태 수렴 · DB unique/check/conditional update로 최종 불변식 보호

적용 프로젝트

StockRush · Member Event Consistency

🔄 실패 복구 설계

실패를 숨기지 않고 retry, DLQ, replay, fallback처럼 추적하고 다시 처리할 수 있는 상태로 남겼습니다.

실패 상태 기록 → 복구 경로 실행 → 수렴 확인

적용 결과

Kafka/outbox 중단 뒤 재발행과 역등 재처리 · LLM fallback, CDC replay, 이벤트 requeue 경로

적용 프로젝트

StockRush · AI Gateway · CDC Data Platform · Fashion Personalization

🛡️ 보안·정책 경계

인증, 권한, 할당량과 provider 정책을 도메인 로직 밖의 명시적인 진입 경계에서 적용했습니다.

신뢰 경계 설정 → 정책 적용 → 권한 검증

적용 결과

OIDC/JWT Gateway와 리소스 소유권 검사 · 검색 전 권한 필터와 LLM policy pipeline

적용 프로젝트

StockRush · Enterprise Policy RAG · AI Gateway

🗃️ 데이터·AI 파이프라인

원천 이벤트가 검색 답변, 분석 mart, 추천 snapshot으로 변환되는 전 과정을 추적 가능하게 설계했습니다.

원천 수집 → 변환·저장 → 계보·품질 확인

적용 결과

citation/refusal을 포함한 RAG 응답 흐름 · CDC lineage와 행동 기반 추천 snapshot

적용 프로젝트

Enterprise Policy RAG · CDC Data Platform · Fashion Personalization

📊 관측·운영 제어

장애와 품질을 확인할 수 있도록 trace, lag, cost, DLQ, freshness를 API와 화면에 노출했습니다.

signal 기록 → API·화면 노출 → 이상 상태 판단

적용 결과

LLM latency-token-cost와 request trace · connector lag-quality SLA-batch freshness report

적용 프로젝트

Enterprise Policy RAG · AI Gateway · CDC Data Platform · Fashion Personalization

🧪 재현 가능한 검증

동시성, 중복, 장애, 정적 데모를 반복 실행 가능한 smoke, integration, regression으로 고정했습니다.

시나리오 고정 → 자동 실행 → 회귀 기준 유지

적용 결과

Docker Compose와 Testcontainers 인프라 검증 · pytest, offline regression, audit runner 자동화

적용 프로젝트

StockRush · Member Event Consistency · AI Gateway · CDC Data Platform

01

StockRush

분산 커머스 / 주문 상태
수령 / 이벤트 기반
백엔드

담당 범위

개인 프로젝트 / 설계,
구현, 검증

핵심 역량

분산 트랜잭션
이벤트 수령
Gateway 보안
장애 복구

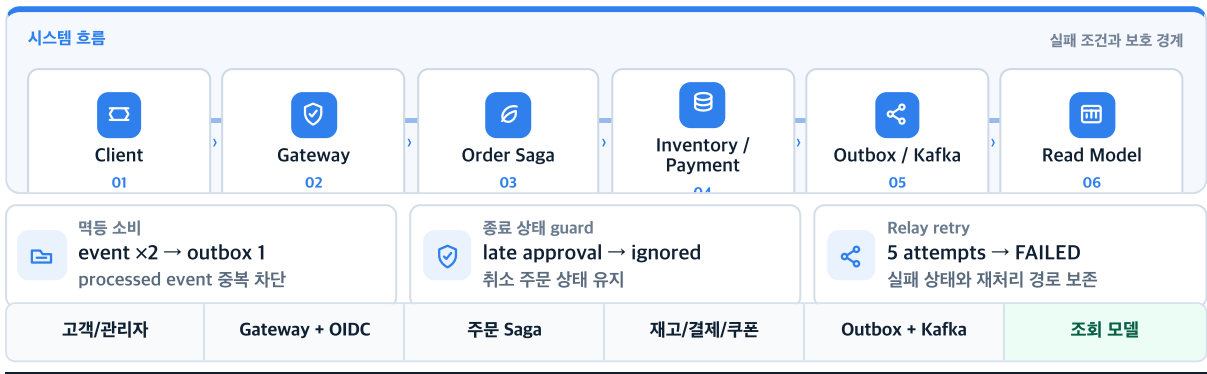
주요 기술

- Spring Boot
- Kafka
- PostgreSQL
- Keycloak

문제 · 설계 · 실패 조건 · 검증 결과

결제 지연과 이벤트 중복 뒤에도 주문 상태를 수렴시키는 커머스

한정 재고 주문에서 결제 지연·이벤트 중복·Kafka 중단이 겹쳐도 주문을 취소·확정 상태로 수렴시키도록 Saga, Outbox, 멍든 소비자를 구현했습니다.



01 해결한 문제 재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남
동시 주문, 결제 실패와 지연, Kafka 중단, 이벤트 재처리 상황에서는 주문 상태와 재고 수량, 결제 결과, 조회 모델이 서로 다른 속도로 바뀝니다. 서비스 경계와 복구 흐름을 코드와 시나리오로 분리해 최종 상태 수렴을 검증했습니다.

02 핵심 설계 판단		
Saga orchestration 재고·결제·쿠폰·출고 상태 전이를 주문 서비스에서 조율	01 Reliable publish DB 변경과 outbox row를 함께 저장해 발행 실패를 재처리	02 Trust boundary Gateway JWT와 서비스 내부 리소스 소유권 검사를 분리
	03	

03 대표 코드 근거 **04 검증 결과**

```
PersistentCreateOrderService.create
파일 services/order-service/src/main/java/com/stockrush/order/application/PersistentCreateOrderService.java

boolean saved = orderRepository.saveIfAbsent(result.order(),
command.idempotencyKey());
if (!saved) {
    return CreateOrderResult.replayed(replayOrder(command.idempotencyKey(),
command.memberId()));
}
outboxEventRepository.save(result.outboxEvent());

보호 규칙주문 저장과 outbox 기록을 같은 트랜잭션에 두고 멍든 충돌은 기존 주문 재조회로 수렴시킵니다.
연결 테스트 services/order-
service/src/test/java/com/stockrush/order/infra/persistence/PersistentCreateOrderServiceIntegrationTest.java
PersistentCreateOrderServiceIntegrationTest.persist_order_items_and_pending_outbox_event_together

OrderSagaEventHandler.transitionOrder
파일 services/order-service/src/main/java/com/stockrush/order/application/OrderSagaEventHandler.java

where order_id = :orderId
and status not in ('CANCELLED', 'CONFIRMED')

return updated == 1;

보호 규칙종료 상태 주문의 추가 전이를 SQL에서 차단해 늦은 이벤트가 취소·확정 주문을 되살리지 못하게 합니다.
연결 테스트 services/order-service/src/test/java/com/stockrush/order/application/OrderSagaEventHandlerIntegrationTest.java
OrderSagaEventHandlerIntegrationTest.ignores_payment_authorized_for_cancelled_order

OutboxRelayService.claimPending
파일 services/order-service/src/main/java/com/stockrush/order/infra/outbox/OutboxRelayService.java

where status = 'PENDING'
and (next_retry_at is null or next_retry_at <= now())
order by created_at, id
limit :batchSize
for update skip locked

보호 규칙재시도 가능 이벤트만 잠금 선점해 여러 relay의 중복 발행을 피하고 소진 실패를 FAILED로 남깁니다.
연결 테스트 services/order-service/src/test/java/com/stockrush/order/infra/outbox/OutboxRelayServiceIntegrationTest.java
OutboxRelayServiceIntegrationTest.marks_failed_when_publish_retry_count_is_exhausted
```

- OrderSagaEventHandlerIntegrationTest가 동일 이벤트 2회 처리에도 outbox 1건만 남도록 고정합니다.
- 취소 완료 뒤 늦게 도착한 결제 승인 이벤트가 주문을 되살리지 않는 상태 guard를 검증합니다.
- OutboxRelayServiceIntegrationTest가 retry 5회 소진 뒤 FAILED 전이와 재처리 상태를 확인합니다.

구현 결과 한정 판매 커머스의 동시 주문, 결제 실패/지연, 이벤트 발행 실패, 중복 처리, 운영자 보정을 재현 가능한 흐름으로 묶었습니다.	확인한 범위와 남은 경계 코드와 통합 테스트로 멍든 소비·종료 상태 guard·relay retry를 다뤘으며 실결제·다중 리전·운영 규모 부하는 포함하지 않았습니다.
--	--

03

Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

담당 범위

2026.05-2026.06 / 개인 프로젝트 / Spring Boot 백엔드, 인프라 비교, 검증 대시보드

핵심 역량

동시성 제어
업무 불변식
Redis/RabbitMQ
Testcontainers

주요 기술

- Spring Boot
- PostgreSQL
- Redis
- RabbitMQ

문제 · 설계 · 실패 조건 · 검증 결과

보상·쿠폰·포인트 불변식마다 다른 동시성 제어를 적용한 백엔드

첫 로그인 보상·쿠폰 발급·포인트 차감의 불변식을 PostgreSQL에서 최종 보호하고, Redis lock과 RabbitMQ 단일 worker의 적용 차이를 동시 요청으로 비교했습니다.

전략 비교

실패 조건과 보호 경계

Scenario API
01

DB Baseline
02

Redis + DB
03

RabbitMQ Lane
04

Outbox
05

Infra Tests
06

최초 보상
8 requests → 1 reward
7 rejected · duplicate 0

포인트 차감
100 - (2 × 60) → 40
1 success · negative 0

쿠폰 용량
8 requests / capacity 3
3 issued · 5 rejected

이벤트 시나리오

불변식 식별

DB guard

Redis/RabbitMQ 비교

Outbox

비교 리포트

01 해결한 문제

중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액
모든 회원 이벤트를 하나의 memberId lock으로 묶으면 hot key 병목이 커지고, 아무 제어 없이 처리하면 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액, 후속 이벤트 누락이 발생합니다.

02 핵심 설계 판단

01 DB is final guard

unique-check-조건부 갱신-row lock으로 최종 불변식 보호

02 Lock by invariant

Redis lock을 전역 잠금이 아닌 경합 완화 수단으로 사용

03 Queue hot campaigns

RabbitMQ 단일 worker로 hot campaign 직렬화 비교

03 대표 코드 근거

SqlCouponCampaignRepository.issueWithCapacityGuard

파일 backend/src/main/java/com/example/consistency/coupon/SqlCouponCampaignRepository.java

where id = ?
and status = 'ACTIVE'
and issued_count < capacity
for update

보호 규칙캠페인 행 잠금과 용량 조건을 한 SQL 흐름에서 판정해 동시 요청의 초과 발급을 차단합니다.

연결 테스트 backend/src/test/java/com/example/consistency/coupon/SqlCouponCampaignRepositoryTest.java
SqlCouponCampaignRepositoryTest.issueWithCapacityGuardUsesSingleStatementWithCampaignRowLock

SqlPointSpendRepository.tryDebit

파일 backend/src/main/java/com/example/consistency/point/SqlPointSpendRepository.java

version = version + 1,
updated_at = now()
where member_id = ?
and balance >= ?

보호 규칙잔액이 지출액 이상일 때만 원자적으로 차감해 동시 요청에서도 음수 잔액을 허용하지 않습니다.

연결 테스트 backend/src/test/java/com/example/consistency/point/SqlPointSpendRepositoryTest.java
SqlPointSpendRepositoryTest.conditionalDebitKeepsBalanceNonNegative

CouponCampaignRabbitMqWorker.handle

파일 backend/src/main/java/com/example/consistency/web/CouponCampaignRabbitMqWorker.java

@RabbitListener(queues = CouponCampaignRabbitMqConfiguration.COMMAND_QUEUE,
concurrency = "1")
public void handle(CouponCampaignRabbitMqCommand command) {
if (!tracker.isActive(command.operationId())) {
return;
}
}

보호 규칙단일 consumer로 hot campaign을 직렬화하고 종료된 실행의 메시지를 무시하며 재시도·DLQ 경로를 분리합니다.

연결 테스트 backend/src/test/java/com/example/consistency/integration/MvpLiveInfrastructureIT.java
MvpLiveInfrastructureIT.rabbitMqCouponCampaignRouteRunsOnlyWhenLiveDependenciesAreHealthy

구현 결과

DB guard를 최종 보호 장치로 두고 Redis lock, RabbitMQ lane, Outbox 책임을 업무 불변식별로 분리했습니다.

확인한 범위와 남은 경계

PostgreSQL·Redis·RabbitMQ 통합 경로를 Testcontainers 시나리오로 고정했으며 운영 규모 부하와 장시간 장애 복구 SLO는 포함하지 않았습니다.

코드 저장소 · 설계·검증 문서

05

02

Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

담당 범위

2026.05-2026.06 / 개인 프로젝트 / AI 백엔드, 운영 콘솔, 검증 문서화

핵심 역량

RAG
권한 필터
근거 답변
운영 관측

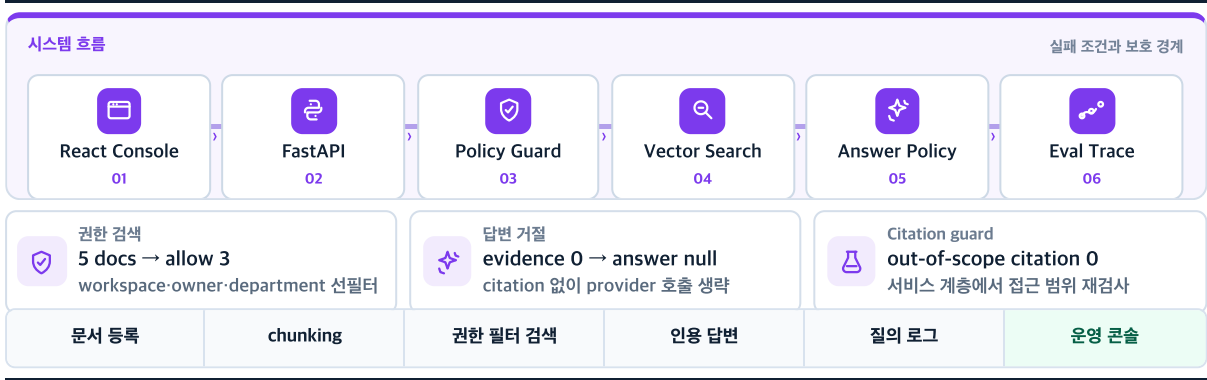
주요 기술

- Python API
- React/Vite
- PostgreSQL/pgvector
- OpenAI 선택 경로

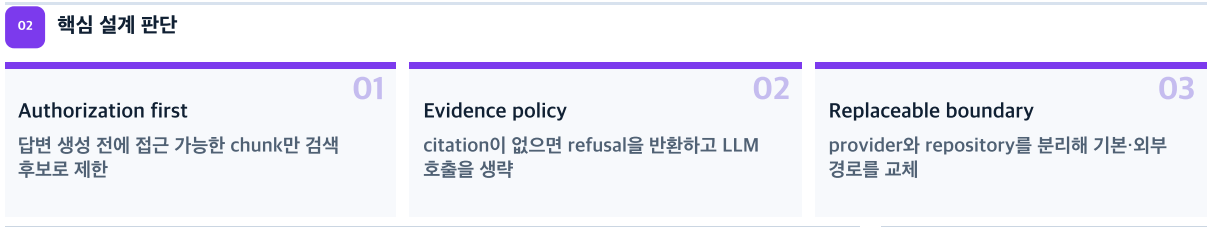
문제 · 설계 · 실패 조건 · 검증 결과

비인가 검색과 근거 없는 생성을 답변 전에 차단하는 정책 RAG

권한 밖 문서가 검색·인용되지 않고 근거가 없으면 LLM을 호출하지 않도록, 권한 선필터와 citation/refusal 정책을 답변 경로에 넣었습니다.



01 해결한 문제 비인가 문서 노출과 근거 없는 답변 생성
기업 내부 정책 RAG는 검색 품질만으로 부족합니다. 사용자가 볼 수 없는 문서가 검색 후보에 섞이지 않아야 하고, 답변에는 근거가 남아야 하며, 근거가 부족한 경우 거절 응답을 반환해야 합니다.



03 대표 코드 근거

```
RetrievalService.retrieve

파일 app/retrieval.py

access_reason = _access_reason(chunk, query)
if access_reason is None:
    continue

보호 규칙저장소 필터 뒤에도 워크스페이스·소유자·부서 가시성을 다시 판정해 비인가 결과를 제거합니다.

연결 테스트 tests/test_retrieval_permissions.py
test_retrieval_allows_public_owner_and_department_documents_only

AnswerService.answer

파일 app/answer.py

if not citations:
    return AnswerResponse(
        query=query.query,
        answer=None,
        citations=[],
        refusal_reason="insufficient_evidence",
    )

보호 규칙인용 근거가 없으면 provider 호출 없이 명시적인 거절 응답을 반환해 근거 없는 답변을 막습니다.

연결 테스트 tests/test_answer_api.py
test_answer_api_refuses_when_no_evidence_is_available

run_eval

파일 app/evaluation.py

cases = [_run_case(services, request.workspace_id, case) for case in GOLDEN_CASES]
retrieval_hit_rate = _rate(case.retrieval_hit for case in cases)
citation_coverage = _rate(case.citation_covered for case in cases)

보호 규칙고정 평가 케이스를 실제 답변 경로로 실행해 검색 적중률과 인용 커버리지를 함께 계산합니다.

연결 테스트 tests/test_eval_api.py
test_eval_run_api_returns_retrieval_and_citation_metrics
```

04 검증 결과

- 권한 검색 테스트에서 문서 5개 중 허용 범위 3개만 반환하고 다른 workspace 결과는 0건으로 고정합니다.
- 근거가 없으면 answer=null, citations=[]를 반환하고 provider 호출을 생략합니다.
- 권한 밖 보안 문서가 검색 결과와 citation에 포함되지 않는 API 시나리오를 검증합니다.

구현 결과 권한 밖 문서를 검색·인용에서 차단하고, 근거가 없으면 provider 호출 없이 거절 응답을 반환합니다.	확인한 범위와 남은 경계 기본 경로는 메모리 저장소와 정적 데모로 검증했으며 PostgreSQL/pgvector·OpenAI live 경로와 운영 인증은 선택 검증으로 분리했습니다.
--	--

04

AI Gateway

LLM Gateway /
라우팅 / 비용 보호 /
캐시 / 폴백 / 관측

담당 범위

2026.06 / 개인
프로젝트 / AI 백엔드
· LLM 플랫폼 설계,
구현, 검증

핵심 역량

LLM Gateway
비용 보호
캐시/폴백
WebFlux

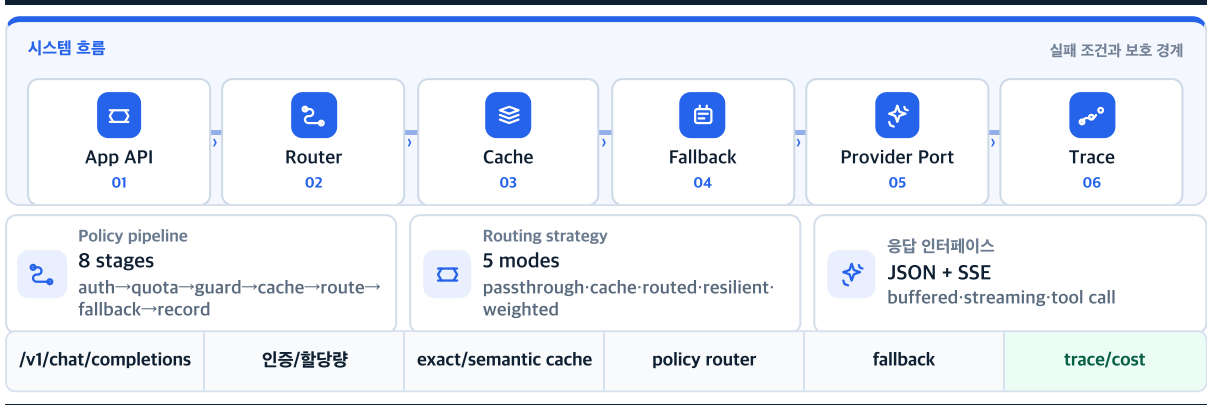
주요 기술

- Java 21
- Spring Boot WebFlux
- Redis
- PostgreSQL/pgvector

문제 · 설계 · 실패 조건 · 검증 결과

예산 초과와 provider 실패를 호출 전에 통제하는 LLM Gateway

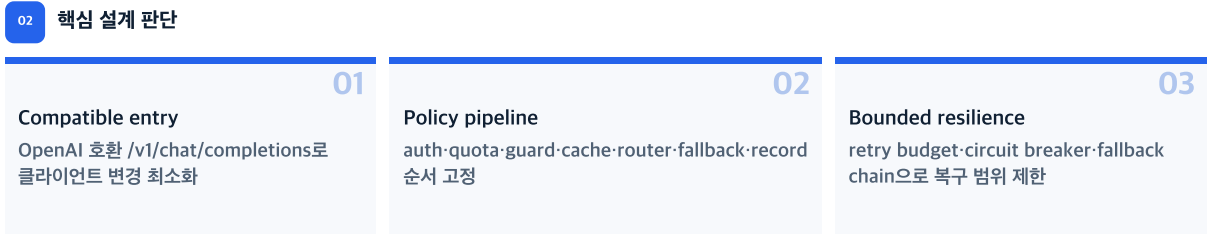
서비스마다 흩어진 LLM 인증·예산·캐시·라우팅·폴백을 OpenAI 호환 WebFlux Gateway의 한 요청 파이프라인으로 통합했습니다.



01 해결한 문제

앱마다 중복되는 LLM 정책과 provider 장애 처리

LLM 기능을 여러 애플리케이션에 붙이면 provider SDK, 장애 대응, 토큰·비용 예산, 캐시, 보호 규칙, 요청 관측이 서비스마다 따로 구현됩니다. Gateway가 공통 정책과 관측을 처리하도록 구조를 모았습니다.



03 대표 코드 근거

GatewayPipeline.execute

파일 backend/src/main/java/com/example/gateway/api/GatewayPipeline.java

```
QuotaOutcome quota = quotaGuard.evaluate(request, estimatedTokens);
if (quota != QuotaOutcome.ALLOWED) {
    return rejected(request, mode, quota, elapsed(start));
}
```

보호 규칙예산 토큰으로 tenant 할당량을 먼저 검사해 초과 요청을 provider 호출 전에 종료합니다.

연결 테스트 backend/src/test/java/com/example/gateway/observability/RequestLogStoreTest.java
RequestLogStoreTest.quotaRejectPathCreatesRowWithNullProviderAndOutcome

TwoStageCache.lookup

파일 backend/src/main/java/com/example/gateway/cache/TwoStageCache.java

```
if (exactLookup.hit()) {
    return exactLookup;
}
return semantic.lookup(request);
```

보호 규칙동일 요청은 exact cache에서 반환하고 miss일 때만 semantic cache로 확장합니다.

연결 테스트 backend/src/test/java/com/example/gateway/cache/TwoStageCacheTest.java
TwoStageCacheTest.differentlyWordedSimilarPromptFallsThroughToSemanticHit

FallbackChain.dispatch

파일 backend/src/main/java/com/example/gateway/resilience/FallbackChain.java

```
lastErrorType = RETRY_BUDGET_EXHAUSTED;
events.add(FallbackEvent.failed(attempt, candidate.provider(),
candidate.model(), lastErrorType));
return FallbackResult.ofFailure(attempt, lastErrorType, events);
```

보호 규칙재시도 예산이 소진되면 다음 provider 호출을 차단하고 관측 가능한 실패 이벤트를 남깁니다.

연결 테스트 backend/src/test/java/com/example/gateway/resilience/FallbackChainTest.java
FallbackChainTest.exhaustedBudgetBlocksFallbackWithoutCallingProvider

04 검증 결과

- ChatCompletionController 테스트가 JSON buffered 응답과 SSE streaming 경로를 분리해 검증합니다.
- 인증 누락 401, 잘못된 요청 400, quota·budget 보호 규칙을 API 테스트로 고정합니다.
- PolicyRouter와 pipeline 테스트가 weighted routing, tool-call passthrough, cache 우회와 fallback 순서를 확인합니다.

구현 결과 앱별 LLM 호출 코드에 흩어질 수 있는 라우팅, 비용 보호, 캐시, 폴백, 관측 기능을 Gateway 계층으로 모았습니다.	확인한 범위와 남은 경계 기본 검증은 고정 응답 provider와 메모리 저장소를 사용했으며 실제 provider 전송·운영 저장소·클라우드 배포는 포함하지 않았습니다.
---	--

05

CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

담당 범위

2026.06 / 개인 프로젝트 / CDC 데이터 플랫폼 백엔드 설계, 구현, 검증

핵심 역량

CDC
Kafka 복구
Lineage
Lakehouse

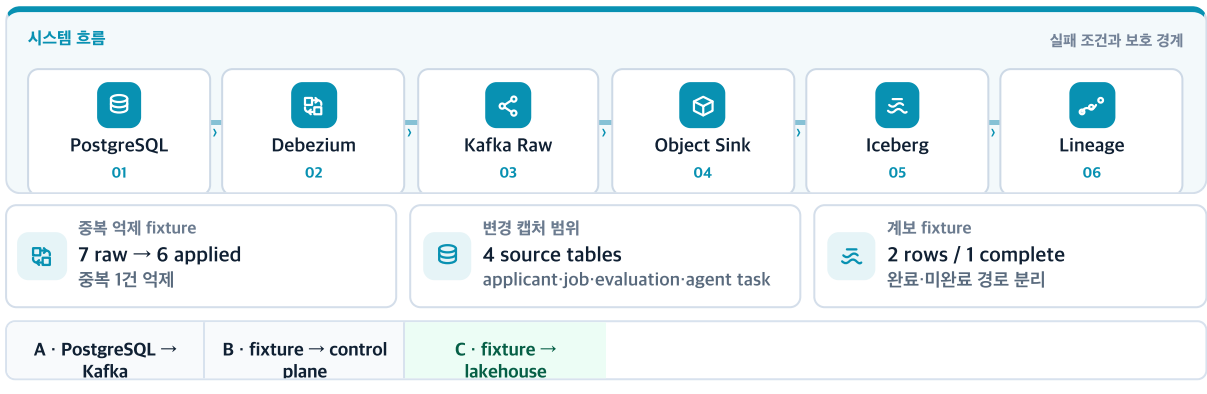
주요 기술

- Spring Boot
- PostgreSQL
- Debezium
- Kafka

문제 · 설계 · 실패 조건 · 검증 결과

변경 캡처·복구·저장을 독립 경계에서 검증한 CDC 플랫폼

PostgreSQL 변경 캡처, canonical event 제어, lakehouse 저장을 세 독립 실험으로 구현했습니다. 각 경계에서 source metadata, 중복 ledger, replay 상태와 lineage fixture를 검증했습니다.



01 해결한 문제 중복 CDC, 복구 이력 손실, 연결되지 않은 구간의 보장 과장

CDC 이벤트는 connector 재시작과 재전달로 중복될 수 있고 sink 실패 뒤 복구 이력이 사라질 수 있습니다. runtime-제어면·lakehouse가 아직 자동 연결되지 않은 상태에서도 각 경계의 보호 규칙과 검증 결과를 구분할 필요가 있었습니다.

01	02	03
Metadata-derived ID LSN·offset를 event ID와 replay·lineage 기준으로 사용	복구 상태 분리 raw·canonical·retry·DLQ·replay 상태를 분리해 원인과 재처리를 추적	독립 검증 경계 runtime·control plane·lakehouse 사이의 미연결 구간을 명시하고 각 fixture 결과만 주장

03 대표 코드 근거 **04 검증 결과**

```
CanonicalIngestService.ingestRawEnvelope
파일 backend/src/main/java/com/example/cdcplatform/event/CanonicalIngestService.java

DebeziumEnvelope envelope = DebeziumEnvelope.parse(rawEnvelopeJson);
CanonicalCdcEvent event = CanonicalCdcEvent.fromEnvelope(envelope);
String sourceOffsetJson = sourceOffsetJson(envelope);
boolean created = canonicalEventPublisher.recordCanonicalEvent(event, sourceOffsetJson);

보호 규칙Debezium 원본을 표준 이벤트로 정규화하면서 source offset을 보존하고 ledger 결과로 중복을 판정합니다.
연결 테스트 backend/src/test/java/com/example/cdcplatform/event/CanonicalIngestServiceTest.java
CanonicalIngestServiceTest.duplicateRawEnvelopeDoesNotIncreaseCanonicalEventCount

ReplayRequestService.requestReplay
파일 backend/src/main/java/com/example/cdcplatform/replay/ReplayRequestService.java

DlqEventRepository.DlqEvent dlqEvent = dlqEventRepository.findById(dlqEventId)
.orElseThrow(() -> new IllegalArgumentException("DLQ event not found: " + dlqEventId));
String requestId = "replay_" + UUID.randomUUID();

보호 규칙존재하는 DLQ 이벤트만 replay 대상으로 허용하고 요청 추적용 고유 ID를 생성합니다.
연결 테스트 backend/src/test/java/com/example/cdcplatform/replay/ReplayRequestServiceTest.java
ReplayRequestServiceTest.createReplayRequestAndMarksDlqEventForReplay

IcebergJavaEngine.append
파일 tools/iceberg-engine/src/main/java/com/example/cdcplatform/tools/IcebergJavaEngine.java

DataFile dataFile = writer.toDataFile();
table.newAppend().appendFile(dataFile).commit();
table.refresh();
Snapshot snapshot = table.currentSnapshot();

보호 규칙Parquet data file을 Iceberg append로 commit하고 현재 snapshot을 다시 조회해 적재 이력을 남깁니다.
연결 테스트 tools/tests/test_lakehouse_smoke.py
LakehouseSmokeTest.test_iceberg_engine_append_passes_insert_sql_and_writes_data_file
```

- local lineage fixture가 raw event 7건 중 6건을 적용하고 duplicate 1건을 억제합니다.
- applicant·job·evaluation·agent task 4개 source table의 metadata와 lineage key를 보존합니다.
- lineage fixture 2개 중 1개 complete path를 식별하고 incomplete 경로를 분리해 보고합니다.

구현 결과 변경 캡처에서는 operation·LSN·offset를, 제어면에서는 event ID·ledger·복구 상태를, lakehouse fixture에서는 object key와 lineage 완료 여부를 각각 추적합니다.

확인한 범위와 남은 경계 CDC runtime, control plane, lakehouse는 독립 프로토타입입니다. A의 Kafka topic을 B가 상시 소비하거나 B의 결과가 C로 자동 전달되는 경로와 AWS S3/Athena·Trino 실행은 검증하지 않았습니다.

06

Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 batch / 이벤트 처리

담당 범위

2026.07 / 개인 프로젝트 / Python 백엔드, 이벤트 처리, 추천 batch, 운영 전환 설계

핵심 역량

Python API
개인화 추천
이벤트 처리
Batch workflow

주요 기술

- Python 3.11+
- optional FastAPI
- pytest
- event processing

문제 · 설계 · 실패 조건 · 검증 결과

실패 이벤트를 격리하고 추천 최신성을 추적하는 개인화 백엔드

중복·실패 행동 이벤트가 사용자 프로필을 오염시키지 않도록 먹등 처리와 DLQ를 적용하고, 결정적 추천 점수와 snapshot freshness를 함께 기록했습니다.

시스템 흐름

실패 조건과 보호 경계

Catalog01

Event Processor02

Profile03

Ranker04

Snapshot05

Admin Report06

실패 이벤트
3 failures → DLQ
새 idempotency key로 requeue

추천 fixture
6 products
brand-size-price-stock scoring

Batch snapshot
snapshots ≥ 2 · stale 0
active user freshness assertion

상품 catalog

사용자 이벤트

idempotency/retry

개인화 profile

추천 batch

관리자 report

01 해결한 문제

중복·실패 이벤트가 프로필과 추천 snapshot을 오염함

패션 커머스 추천은 브랜드, 카테고리, 태그, 사이즈, 가격대, 재고 같은 catalog feature와 사용자 행동 이벤트가 함께 반영되어야 합니다. 중복 이벤트나 처리 실패가 profile을 오염시키지 않도록 이벤트 안정성, batch snapshot, 관리자 리포트를 하나의 백엔드 흐름으로 분리했습니다.

02 핵심 설계 판단

01

Dependency-free core
외부 설치 없이 domain·service 흐름을 검증

02

Failure isolation
idempotency·retry
budget·DLQ·requeue로 오염 차단

03

Realtime vs batch
profile 갱신과 materialized snapshot
생성을 분리

03 대표 코드 근거

04 검증 결과

apply_event_to_profile

파일 src/fashion_personalization/recommendation.py

selected_size = event.payload.get("size")
if isinstance(selected_size, str) and selected_size in product.sizes:
 _add_weight(profile.size_preferences, selected_size, max(weight, 0.1))

보호 규칙선택한 값이 실제 상품 옵션에 있을 때만 사이즈 선호에 반영해 잘못된 프로필 신호를 막습니다.

연결 테스트 tests/test_recommendation_engine.py
test_size_preference_contributes_to_ranking

rank_products

파일 src/fashion_personalization/recommendation.py

score, reasons = _score_product(profile, product)
recommendations.append(Recommendation(product_id=product.product_id,
score=round(score, 4), reasons=tuple(reasons)))
return sorted(recommendations, key=lambda item: (-item.score,
item.product_id))[:limit]

보호 규칙점수와 추천 이유를 함께 저장하고 상품 ID tie-break로 결과 순서를 결정적으로 유지합니다.

연결 테스트 tests/test_recommendation_engine.py
test_personal_ranking_prioritizes_user_preferences

RecommendationBatchWorkflow.refresh_snapshots

파일 src/fashion_personalization/batch.py

generated_at=utc_now(),
source_event_count=profile.event_count,
)
self.store.save_snapshot(snapshot)

보호 규칙snapshot에 생성 시각과 반영 이벤트 수를 기록해 결과 최신성과 event watermark를 추적합니다.

연결 테스트 tests/test_batch_workflow.py
test_batch_refresh_writes_materialized_snapshots

- EventProcessor 테스트가 동일 idempotency key를 한 번만 반영하고 3회 실패 뒤 DLQ로 격리합니다.
- 추천 테스트가 6개 상품 fixture에서 브랜드·사이즈·가격·재고 조건을 deterministic ranking에 반영합니다.
- Batch workflow 테스트가 active user별 snapshot을 2개 이상 만들고 stale snapshot 0을 확인합니다.

구현 결과

중복 이벤트를 한 번만 반영하고, 3회 실패를 DLQ로 격리하며 추천 snapshot과 freshness 지표를 생성합니다.

확인한 범위와 남은 경계

검증 구현은 인메모리 저장소를 사용하며 FastAPI는 선택 adapter입니다. PostgreSQL·SQS·EventBridge·S3·AWS 배포는 포함하지 않았습니다.

손찬양 · 06

로컬 코드 저장소 · 설계·검증 문서

09

기술 선택과 검증 결과

무엇이 깨지는가 · 어떻게 막았는가 · 무엇을 확인했는가

시스템	실패 조건	보호 설계	로컬에서 확인한 결과
01 StockRush	재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남	Saga 상태 전이, 트랜잭션 Outbox, 멍등 소비, 종료 상태 SQL guard	동일 이벤트 2회에도 outbox 1건, 늦은 결제 승인 무시, relay 5회 실패 뒤 FAILED
03 Member Event Consistency	중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액	DB 제약·조건부 갱신을 최종 guard로 두고 Redis·RabbitMQ를 경합별 비교	동시 8건→보상 1건, 60원 차감 2건→잔액 40, 용량 3→정확히 3건 발급
02 Enterprise Policy RAG	비인가 문서 노출과 근거 없는 답변 생성	검색 전 권한 필터, citation 재검사, 근거 없음 refusal, 질의 로그	문서 5개 중 허용 3개만 검색, 근거 0이면 answer=null·provider 호출 0회
04 AI Gateway	앱마다 중복되는 LLM 정책과 provider 장애 처리	API key·quota, 2단계 cache, 정책 router, 제한된 retry·fallback, trace	8단계 파이프라인과 5개 라우팅 모드, JSON·SSE·401·400 경로를 회귀 테스트로 고정
05 CDC Data Platform	중복 CDC, 복구 이력 손실, 연결되지 않은 구간의 보장 과장	독립 runtime·control plane·lakehouse 실험, event ID, ledger, replay 상태	raw 7건 중 6건 적용·중복 1건 억제, 4개 source table과 mart lineage 추적
06 Fashion Personalization Platform	중복·실패 이벤트가 프로필과 추천 snapshot을 오염함	멍등 event processor, retry·DLQ, 결정적 ranking, batch snapshot	동일 key 1회 반영, 3회 실패 뒤 DLQ, 6개 상품 ranking과 snapshot 2개 이상·stale 0

아직 확인하지 않은 운영 범위

아래 항목은 구현 결과나 운영 성과로 주장하지 않습니다.

01 · StockRush 실결제 연동, 다중 리전, 운영 규모 부하와 relay HA	03 · Member Event Consistency 운영 규모 부하, 장시간 Redis/RabbitMQ 장애 복구와 실서비스 SLO	02 · Enterprise Policy RAG 운영 OIDC, 관리형 PostgreSQL/pgvector, OpenAI live 호출과 배포 smoke
04 · AI Gateway 실제 OpenAI/Anthropic 전송, 운영 Redis/PostgreSQL, 클라우드 배포와 부하	05 · CDC Data Platform AWS S3/Athena, 장기 Kafka Connect, Trino, 운영 규모 부하와 다중 리전	06 · Fashion Personalization Platform PostgreSQL·SQS/EventBridge·S3 adapter, AWS 배포, 실제 ML·A/B 성과